

Architecture Decision Record

ADR-001

PostgreSQL como base de dados principal

Status: Aceite

Data	2026-05-08
Decisor	Equipa de Arquitectura
Projecto	Payment Service — Moçambique
Versão	1.0

Contexto

O sistema de pagamentos precisa de persistência transaccional com garantias ACID. Os pagamentos envolvem operações de débito/crédito que não podem ser perdidas nem duplicadas.

O ambiente alvo é Moçambique, onde a infraestrutura pode ser limitada — o banco de dados precisa de ser operável com recursos modestos e ter suporte robusto em ambientes Kubernetes.

❑ *Requisito crítico: um pagamento deve ser gravado inteiramente ou não ser gravado — estados intermédios inconsistentes são inaceitáveis num sistema financeiro.*

Decisão

Usar **PostgreSQL 17** como única base de dados relacional do sistema.

Razões

- **ACID completo** — transacções atómicas garantem que um pagamento ou é gravado inteiramente ou não é gravado, sem estados intermédios inconsistentes.
- **Suporte nativo a UUID** — o campo id é UUID gerado pela JPA, sem colisões e sem expor sequências numéricas previsíveis na API.
- **UNIQUE constraint em idempotency_key** — a unicidade é garantida ao nível da base de dados, não apenas na aplicação, eliminando race conditions em cenários de alta concorrência.

- **Flyway** — migrações versionadas permitem evoluir o schema de forma controlada e reproduzível em todos os ambientes (dev, staging, prod).
- **Ecossistema Java** — integração madura com Spring Data JPA, HikariCP e Testcontainers.
- **Custo operacional** — open-source, bem suportado em Kubernetes via Helm charts e operadores (CloudNativePG).

Alternativas Consideradas

Alternativa	Razão de Rejeição
MySQL	Menor suporte a tipos avançados (JSONB, arrays); comportamento de transacções ligeiramente diferente
MongoDB	Sem transacções multi-documento no modo standalone; modelo de dados financeiros beneficia de schema rígido
H2 (in-memory)	Apenas para testes; sem suporte a produção

Consequências

Positivas

- Garantias ACID nativas para todas as operações de pagamento
- Schema versionado com Flyway — rollback controlado em caso de falha de deploy
- Paridade completa entre ambientes de teste e produção via Testcontainers
- Operadores Kubernetes (CloudNativePG) facilitam backups, failover e upgrades

Negativas / Trade-offs

- A aplicação fica acoplada ao PostgreSQL; migrar para outro RDBMS requer mudança de dialect Hibernate e validação das migrações Flyway
- Requer gestão de estado persistente no Kubernetes (StatefulSet + PVC)

❑ *Os testes de integração usam a mesma imagem postgres:17-alpine via Testcontainers, garantindo paridade exacta entre o ambiente de teste e produção.*

Notas de Implementação

Configuração relevante no schema de migração (Flyway):

```
-- V1__create_payments_table.sql
CREATE TABLE payments (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  idempotency_key VARCHAR(255) NOT NULL,
  amount      NUMERIC(19, 4) NOT NULL,
```

```
currency      VARCHAR(3) NOT NULL DEFAULT 'MZN',
status        VARCHAR(20) NOT NULL DEFAULT 'PENDING',
created_at    TIMESTAMP NOT NULL DEFAULT now(),
updated_at    TIMESTAMP NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX uq_payments_idempotency_key
ON payments (idempotency_key);
```

Referências

- ADR-002 — RabbitMQ para processamento assíncrono
- ADR-003 — Idempotência via Idempotency-Key